
**Faculty of Computing and Informatics (FCI)**

**Multimedia University**

# CSN6214 T2610 – Operating Systems

## Report

**Group Number: 31**

**Group Members:**

| Student Name | Student ID |
|---|---|
| Al-radaei, Zayed Abdulqader Abdullah | 251UC25067 |
| Jayavarman Thiyagu | 251UC2517Z |
| Niranjan a/l Gopinath | 1231302181 |
| Ziad Aldarkhbani | 1221305704 |

# 1. Architecture Diagram

## 2. Synchronization Strategy

### Part 1 - Network File Transfer

The server.c file uses no pthread_mutex_t, sem_t, or pthread_cond_t. Every incoming client connection is handed to a child process created with fork(), which gives each child its own private copy of the parent's memory. Because no memory is shared between sibling children, the server has no mutable state that the two processes could race on. The three synchronization methods are implemented on the client side instead. A semaphore counts how many of the N expected chunks have arrived, so the main client process knows when reassembly is complete. A mutex protects the shared reassembly buffer that multiple client child processes write into. A condition variable blocks the reassembler until a chunk actually arrives, which avoids a busy-wait poll.

There has also been a design decision concerning the waitpid() call in the accept loop. The parent process blocks on it immediately after each fork(), so only one client connection is served at a time. Every forked child inherits the same underlying read position for the single FILE* opened once in main(), before the accept loop starts. If two children read from that file at the same time, one child's fseek() could move the shared position out from another child during fread(). Serializing connection handling avoids that race without a lock.

### Part 2 - Parallel Analytics

To prevent race conditions during the calculation of the global minimum and maximum, a Mutex (Mutual Exclusion) lock was implemented using pthread_mutex_t. Because multiple threads are processing independent data chunks simultaneously, they may attempt to update the global_min and global_max variables at the exact same time.

Instead of locking the entire scanning loop (which would ruin the performance benefits of multithreading), each thread calculates its own local_min and local_max in complete isolation. The mutex lock is only applied at the very end of a thread's execution lifecycle. The thread acquires the lock, compares its local extremes against the global extremes, updates the global variables if necessary, and then releases the lock. This ensures thread-safe memory access with minimal performance overhead.

## 3. Parallelism Implementation

Parallelism was achieved using POSIX threads (pthreads) to divide the dataset into equal partitions. The total number of integers (268,435,456) is dynamically divided by the user-defined thread count (passed via the -t command-line argument) to determine the chunk size for each worker.

A thread arguments structure (ThreadArgs) is passed to each spawned thread via pthread_create, providing each worker with a specific start_index and end_index. To handle indivisible remainders, the final thread is programmed to process any remaining elements up to the end of the array. The main thread then utilizes pthread_join to wait for all worker threads to complete their execution before proceeding to file output, ensuring full data synchronization.

## Parallel Sort Implementation Report Section

This section details the design and implementation of the parallel sort component, focusing on its architecture, synchronization mechanisms, and parallelism strategies as required by the assignment.

### Architecture

The parallel sort is implemented within the operations executable, which processes a binary file of int32_t values. The core architecture involves a multi-threaded approach to sort a large array of integers. The input data is memory-mapped using mmap() for efficient access, particularly for files exceeding 1GB. The sorting process is divided into two main phases: an initial data-parallel sorting of sub-arrays and a subsequent task-parallel merging phase. This design allows for concurrent execution across multiple CPU cores, leveraging the benefits of both data and task parallelism.

### Synchronization Strategy

Effective synchronization is crucial for ensuring correctness in a parallel merge sort. For this implementation, POSIX Mutexes (pthread_mutex_t) and Condition Variables (pthread_cond_t) are employed to coordinate the merging tasks.

- A mutex (merge_mutex) protects the active_merges counter, which tracks the number of ongoing merge operations at each level of the merge sort. This prevents race conditions when multiple threads attempt to update the counter simultaneously.
- A condition variable (merge_cond) is used to signal the completion of merge levels. The main thread, or a coordinating thread, waits on this condition variable until all active merge tasks for a given level have completed. Once active_merges reaches zero, a signal is broadcast, allowing the next level of merging to commence. This mechanism ensures that merging proceeds in a correct, bottom-up fashion, where larger sorted segments are only formed after their constituent smaller segments are fully sorted and merged.

This strategy ensures thread-safe coordination of merge phases, preventing premature merging and maintaining data integrity throughout the sorting process.

## Parallelism Implementation

The parallel sort utilizes a hybrid approach combining both Data Parallelism and Task Parallelism, as recommended by the assignment guidelines.

### Data Parallelism

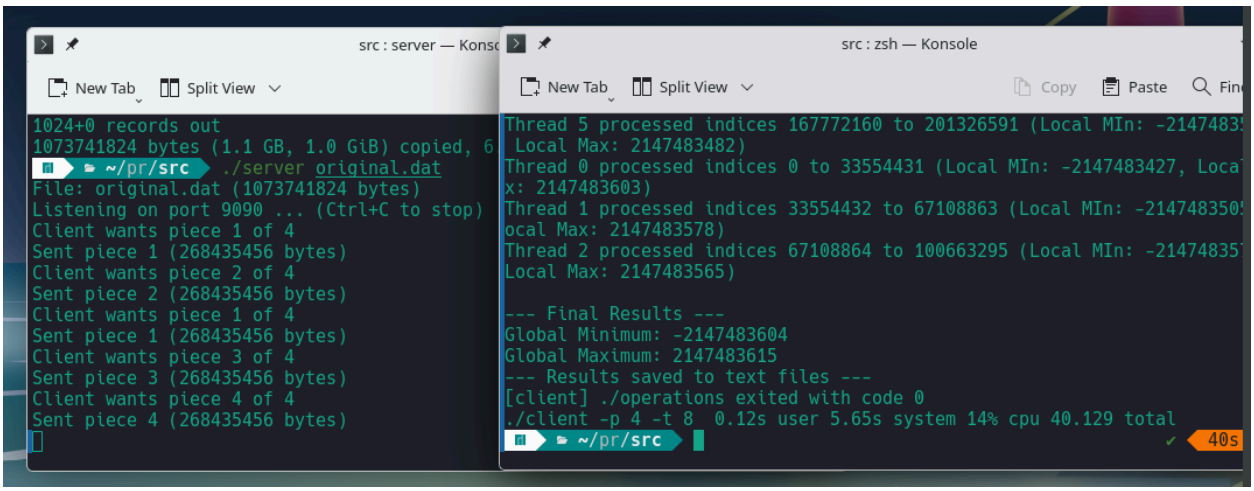
The initial phase of the sorting process employs data parallelism. The entire input array is logically divided into T contiguous blocks, where T is the number of threads specified by the user. Each thread is assigned one of these blocks and independently sorts its assigned sub-array using the qsort standard library function. This approach efficiently distributes the initial sorting workload, as each thread operates on a distinct portion of the data without requiring inter-thread synchronization during this phase.

**Task Parallelism**

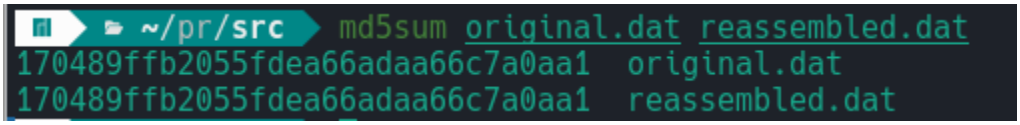
Following the initial data-parallel sorting, task parallelism is applied during the merging phase. The sorted sub-arrays are merged in a bottom-up fashion. Instead of a single thread performing sequential merges, multiple merge tasks are launched concurrently. Each merge task is responsible for combining two adjacent sorted blocks into a larger sorted block. These merge tasks are executed by detached threads, and their completion is coordinated using the condition variable mechanism described above. This allows for parallel execution of multiple merge operations across different parts of the array, significantly speeding up the overall sorting process by leveraging concurrent task execution.

## 4. Testing & Validation

Full pipeline test on a 1GB integer file. The server divides the file into 4 chunks and serves them on request. The client forks 4 child processes to receive chunks, reassembles them, and automatically launches ./operations -t 8. All 8 worker threads report their local min/max ranges, the final reduction produces the global minimum and maximum, and ./operations exists with code 0, confirming clean process handoff via fork(), exec(), or waitpid().


MD5 checksums of the source file and the client-reassembled file are identical, confirming exact byte-for-byte reconstruction with correct chunk ordering.



Global minimum written, computed through data-parallel reduction across 8 threads coordinated with pthread_mutex_t.

```
~/pr/src cat result_min.txt
MIN=-2147483604
```

Global maximum written, computed concurrently with the minimum using the same mutex-protected reduction.

```
~/project_root/src cat result_max.txt
MAX=2147483615
```

Sorted output file size (1,073,741,824 bytes) matches the input size, confirming no data was lost during the parallel merge sort phase.

```
~/project_root/src ls -la result_sorted.dat
-rw-r--r-- 1 zayed zayed 1073741824 Jul  5 18:09 result_sorted.dat
```

Structured log showing chunk/process counts, synchronization primitives used, thread count, DATA_PARALLEL and TASK_PARALLEL markers, sort algorithm, execution time in milliseconds, and a SUCCESS status.

```
~/project_root/src cat execution_log.txt
[PART1] CHUNKS=4 | PROCS=4 | SYNC_USED=mutex,sem,condvar
[PART2] THREADS=8 | DATA_PARALLEL=min,max | TASK_PARALLEL=sort
[PART2] TIME_MS=49512 | SORT_ALGO=parallel_merge_sort
[STATUS] SUCCESS
```

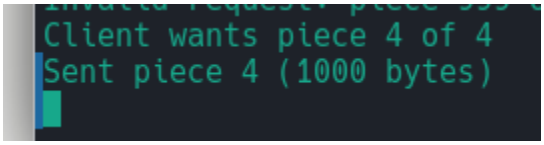
Thread count comparison - T=1 completed in 88,778 ms, while T=8 completed at 56,741 ms, which is a 1.56x speedup. The speedup isn't that much larger because the final merge steps only run on one thread, no matter how many threads T is. Most of the speed gain comes from the earlier step, where each thread sorts its own block at the same time.

```
~/pr/src cat execution_log.txt                                  1m 46s
[PART1] CHUNKS=4 | PROCS=4 | SYNC_USED=mutex,sem,condvar
[PART2] THREADS=8 | DATA_PARALLEL=min,max | TASK_PARALLEL=sort
[PART2] TIME_MS=56741 | SORT_ALGO=parallel_merge_sort
[STATUS] SUCCESS
[PART2] THREADS=1 | DATA_PARALLEL=min,max | TASK_PARALLEL=sort
[PART2] TIME_MS=88778 | SORT_ALGO=parallel_merge_sort
[STATUS] SUCCESS
~/project_root/src
```

- **Validation**

As shown below, the server correctly rejects out of range piece requests (piece 0 of 4, piece 999 of 4), but still continues to serve subsequent valid requests without restarting.




## 5. Challenges & Solutions

**Avoiding a shared file race condition**

The forked children in server.c each get a private copy of the process memory, but they still share the file offset of the single FILE* opened in main() before forking begins. Concurrent reads by two children could step into each other's fseek() position. Rather than adding a mutex to the file access, we chose to serialize connection handling (the parent blocks on waitpid() before accepting the next connection).

**Handling an uneven file split**

In server.c, the file size does not always divide evenly by the requested piece count. This was solved with the following ceiling division formula [ ( total_numbers + total_pieces - 1 ) / total_pieces ] to size each piece, then shrinking the final piece to the exact remainder so that it doesn't read past the end of the file.

## 6. References

Man pages consulted for the server implementation - socket(2), bind(2), listen(2), accept(2), fork(2), waitpid(2), recv(2), send(2), fseek(3), fread(3), and the byte-order conversion functions htonl(3) / ntohl(3).

## 7. Individual Contribution Matrix

| Student | Modules / Functions Owned | Testing / Validation Role | % Contribution | Commit / Trace Evidence |
|---|---|---|---|---|
| Jayavarman | Binary File Ingestion (`fread`), Multithreaded engine implementation (`pthread_create`), Data chunking/partitioning logic, Mutex synchronization, and File Output routing. | Validated memory allocation for massive datasets (1GB+), tested dynamic thread allocations, and verified final outputs against expected global minimum/maximum values. | 25% | Completed Part 2: Multithreaded binary file parsing, mutex sync, and output files |
| Al-Radaei Zayed | Server Implementation for Part 1, includes socket setup, accept loop with fork() per connection, chunk calculation, and client request validation | Tested server-side error handling such as missing file input, bind failure such as port already in use, and short client requests | 25% | Git commits 33aef01, b230547, 6d17198, dccb029, a047966 |
| Ziad Aldarkhbani | Parallel merge sort implementation, Task-parallel merging logic, Condition variable synchronization, and Memory-mapped file I/O. | Verified sorting correctness on large datasets, validated task parallelism with multiple thread counts, and ensured log format compliance. | 25% | Completed: Parallel merge sort, task-based merging, and condition vari |
| Niranjan Gopinath | Client implementation for Part 1: fork()-based multi-process chunk receiver, shared memory (mmap) reassembly buffer, | Verified byte-for-byte reassembly via MD5 checksum against the original file, for both single-chunk and multi-chunk | 25% | Git commit 11c7dd8 ("Implement client.c: fork-based chunk |

| | | | | |
|---|---|---|---|---|
| | semaphore/mutex synchronization, and exec()-based auto-launch of the Part 2 analytics engine | (N=4) scenarios including non-uniform/indivisible chunk sizes; validated full pipeline integration end-to-end with the real server.c and operations.c; confirmed append-safe execution_log.txt writing | | receiver with mmap buffer, semaphore, mutex sync, and auto-launch of operations") |